# Divide-and-Conquer: Closest pair of points on a 2D plane

## Design and Analysis of Algorithms

Dhannya SM

# Problem Definition

- Given a set of $n$ points in a 2D plane, find the pair of points that are closest to each other in terms of Euclidean distance.

# Brute force

- A simple approach is to compare the distance between all pairs of points.
- This takes $O(n^2)$.
- Is there a better solution?

# Divide and Conquer

Algorithm Steps:

- Sort the points by their x-coordinates.
- Divide: Split the points into two halves by a vertical line.
- Conquer: Recursively find the closest pair in both halves, say $(x_{l1}, y_{l1})$ & $(x_{l2}, y_{l2})$ for the left half and $(x_{r1}, y_{r1})$ & $(x_{r2}, y_{r2})$ for the right half.
- Merge: Find the closest pair across the two halves by checking points near the dividing line, say $(x_{m1}, y_{m1})$ & $(x_{m2}, y_{m2})$.
- Among the three pairs - $(x_{l1}, y_{l1})$ & $(x_{l2}, y_{l2})$, $(x_{r1}, y_{r1})$ & $(x_{r2}, y_{r2})$ and $(x_{m1}, y_{m1})$ & $(x_{m2}, y_{m2})$ - pick the closest pair.

# Split the points into two halves by a vertical line

- Given n points $P = \{p_1, p_2, \ldots, p_n\}$, compute $P_x$ which is $P$ sorted by $x$-coordinate and $P_y$, $P$ sorted by $y$-coordinate.
- Divide $P$ by vertical line into equal size sets $Q$ and $R$.
- Need to efficiently compute $Q_x, Q_y, R_x, R_y$
  - $Q_x$ is first half of $P_x$, $R_x$ is second half of $P_x$
  - When splitting $P_x$, note the largest $x$ coordinate in $Q$, $x_Q$
  - Separate $Py$ as $Qy$, $Ry$ by checking $x$-coordinate with $x_Q$
- Basic recursive call is $ClosestPair(Px, Py)$
- Set up recursive calls $ClosestPair(Qx, Qy)$ and $ClosestPair(Rx, Ry)$ for left and right half of $P$ in time $O(n)$.
- How to combine these recursive solutions?

# Middle band using $\delta$

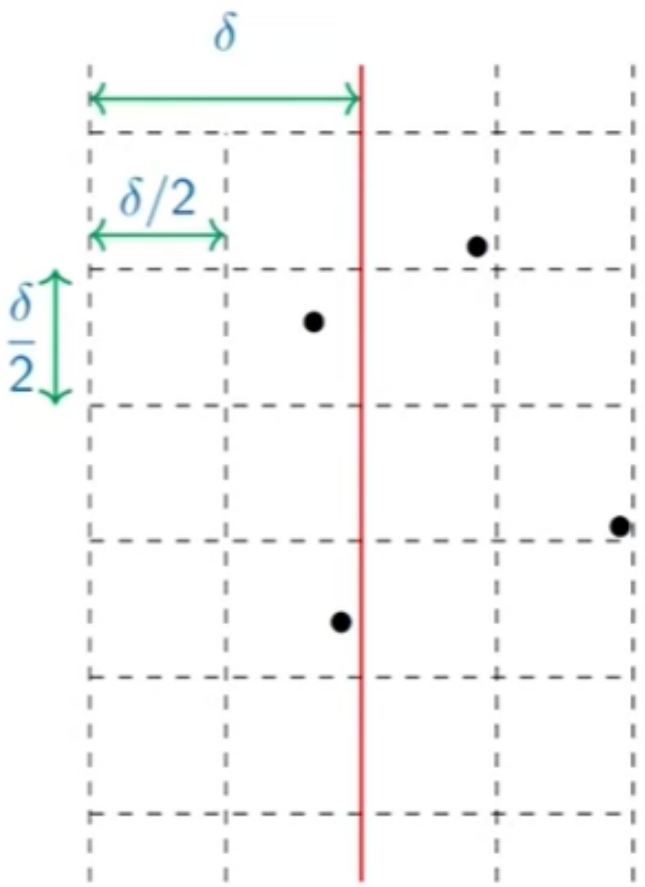


Image and Algorithm Credit: `https://youtu.be/vjjWGpy7DMk`

# Algorithm

Given n points $P = \{p_1, p_2, \ldots, p_n\}$

- $P_x$ is $P$ sorted by $x$-coordinate
- $P_y$ is $P$ sorted by $y$-coordinate
- $Q$ and $R$ are equal size sets by dividing $P$ by a vertical line
- $Q_x, Q_y$ are the first half of $P_x, P_y$ respectively
- $R_x, R_y$ are the second half of $P_x, P_y$ respectively

```
def ClosestPair(Px,Py):

  if len(Px) <= 3:
    compute pairwise distances
    return closest pair and distance

  Construct (Qx,Qy), (Rx,Ry)

  (q1,q2,dQ) = ClosestPair(Qx,Qy)
  (r1,r2,dR) = ClosestPair(Rx,Ry)

  Construct Sy from Qy,Ry
  Scan Sy, find (s1,s2,dS)

  return (q1,q2,dQ), (r1,r2,QR), (s1,s2,dS)
  depending on which of dQ, dR, dS is minimum
```

# Summary

- Splitting the problem
- Solving the cross-over band
- Combining the solutions of subproblems